RanDroid: Structural Similarity Approach for Detecting Ransomware Applications in Android Platform

Abdulrahman Alzahrani*, Ali Alshehri*, Hani Alshahrani*, Raed Alharthi*, Huirong Fu*, Anyi Liu*, Ye Zhu†
*Department of Computer Science and Engineering
Oakland University
Rochester, Michigan 48309
{aalzahrani, aaalshehri, hmalshahrani, rsalharthi, fu, anyiliu}@oakland.edu
†Electrical and Computer Engineering Department
Cleveland State University
Cleveland, OH 44115
y.zhu61@csuohio.edu

***Abstract*—The worldwide epidemic of ransomware monetary gains has grown astonishingly. This crimeware form is emerged to extort innocent users under the threat of locking their devices and/or encrypting their files. To mitigate the growth of ransomware attacks, cybersecurity researchers have proposed various solutions based on the functionalities of those attacks. However, this polymorphic type is kept refined to increase the appearance of new families and survive against mitigation approaches. This paper introduces RanDroid, a new automated lightweight approach for detecting ransomware variants in Android platform by measuring the structural similarity between a set of collected information from an inspected application and a set of predefined threatening information collected from known ransomware variants. Furthermore, RanDroid performs a linguistic analysis on the app's code as well as image textural strings to enhance further revelation. RanDroid was evaluated using 950 ransomware samples. In addition, this approach is capable of extracting threatening messages from samples that use evasion techniques such as sophisticated codes or dynamic payloads.**

***Index Terms*—Ransomware, similarity measurement, threatening images, extorting texts**

I. Introduction

Mobile landscape has hosted a drastically increasing number of applications. Among a variety of mobile operating systems, Android has become the most dominant platform in the market share due to its open source philosophy and expandability [1]. Therefore, Android security is the thriving spotlight that has drawn a significant amount of interest among research society to overcome its threats and alleviate the spread of malice. Furthermore, research efforts investigate security risks and enhance solutions from various perspectives. However, cybercriminals are encouraged to penetrate marketplaces with malicious applications. As a result, the effort of devices' protection is substantially undermined.

Malware family exhibit polymorphic behaviors due to the rampant growth of present technology. For instance, ransomware has constituted as one of the most dangerous cyberthreats to organizations and individuals alike in the recent years. This class of malware elicits immediate monetized returns to their developers. Therefore, it creates a competitive mentality and lucrative opportunities for attackers to maximize their monetized gains and achieve their desired purposes.

Ransomware robbers are capable of pushing malicious activities worldwide. Hence, ransomware attacks are trending upwards, victimizing users using strong and unbreakable encryption techniques [2], [3]. Once an attack is successfully performed, it can cause massive operational disruptions, leading into serious damages to revenues and reputation.

Modern ransomware variants are challenging to be eliminated for two reasons. First, they are easy to obtain or refine. Script kiddie cybercriminals could easily build malware using services such as malware-as-a-service, which is provided for free as a cloud-based service [4], [5]. Second, the operations of ransomware applications can be difficult to distinguish from those of benign applications [6], which fostered malware spread on mobile platforms such as Android.

Our incentive is to address these challenges by proposing RanDroid, a framework that utilizes static and dynamic analyses to detect locking and threatening ransomware variants in Android platform. Unlike other malware types, ransomware notifies victims by displaying notes on their devices. Therefore, RanDroid measures the structural similarity between a set of images collected from an inspected application and a set of threatening images collected from known ransomware variants. Moreover, RanDroid performs a linguistic analysis on the app's contents such as XML layout and classes.dex files to enhance further revelations.

Our major contributions in this paper are summarized as follows:

- Introducing RanDroid, an approach that effectively detects generic locking and threating ransomware variants.

- Utilizing static and dynamic analyses to analyze various types of ransomware applications.
- Classifying samples using structural similarity measurements.
- Evaluating RanDroid on a dataset of 950 ransomware samples.

The rest of this paper is organized as follows. Section II provide a brief background on ransomware threats and techniques that are used in most cases to deploy them. Section III summarizes the existing challenges plaguing security efforts and introduces the problem statement of this paper. Section IV describes the RanDroid architecture and details its implementation. Whereas, the discussion and evaluation of RanDroid is presented in Section V. Other research interests related to eliminating threats and mitigating ransomware attacks are summarized in Section VI. Finally, Section VII provides the future work of RanDroid and concludes this paper.

II. Background

A. Android

Android applications are primarily written in Java and zipped into one package called Android Application Package (APK), which contains a set of files and folders, such as: 1) AndroidManifest.xml where all permissions and requested hardware are declared; 2) Resources folder, which contains all drawable contents such as images, XML layout files, and libraries; and 3) Classes.dex, which is an executable file of the compiled Java code. These files are analyzed using static and/or dynamic analysis approaches to learn about the application contents and identify potential misuse. Moreover, with static analysis, an inspected application is decompiled to its original Java-code format while in dynamic analysis, the application is analyzed during its runtime.

B. Ransomware

Ransomware attacks are classified into two types: locking and cryptographic ransomware [7]. The locking ransomware attacks prevent victims from accessing their devices. This type does not encrypt any files stored on the device [8]. By contrast, cryptographic or crypto-ransomware attacks harm the victims' devices by encrypting the devices' services, files and folders, or even the entire database that the system interacts with. Usually, crypto attacks delete affected files from the devices after encrypting them. After performing a successful attack, a threatening massage is displayed with all needed instructions to render the victim to pay the ransom [9].

After performing a successful infection, the ransomware notifies the victim by displaying a ransom message with instructions on how to pay the ransom within a pre-specified period of time initialized by the ransomware's developers. Affected devices do not permit access until the ransom is paid. Hence, attackers demand payments from the victims for a promise to restore their data. Requested payment method is often requested in an untraceable cryptocurrency such as Bitcoin.

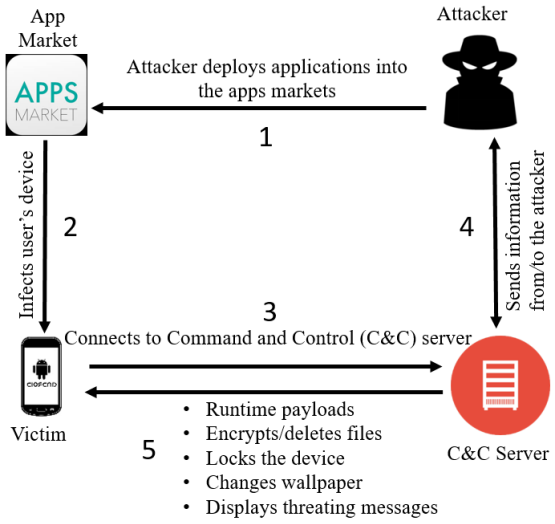

Figure 1: Ransomware attack model

Ransomware developers apply a well-established cryptography technique called hybrid cryptosystem during the attack development [10], [11]. Using such a technique allows attackers to generate one random encryption/decryption key (referred to as a session key or symmetric key) for each targeted object. Subsequently, attackers apply an asymmetric encryption technique using public and private keys generated on the attacker's infrastructure. The public key is embedded to the payload code, which infects users during the runtime, to encrypt the session key. In other words, the asymmetric cryptographic technique is required to encrypt and decrypt the session key regardless of the encrypted content's size. Thus, the strength of ransomware racket is equal to the security of a hybrid cryptosystem [12], [13]. Figure 1 illustrates the attack model of most existing types of ransomware.

III. Problem Statement

With the belief that users download Android applications from potentially untrusted markets, cyber-extorting ransomware victimizations has imposed for detection techniques to prevent damages. Current literature of security efforts in fighting against ransomware attacks to safeguard users' devices can be categorized into three main types of solutions. The first solution consists of analyzing an inspected app's code to distinguish malicious behaviors using static and dynamic analyses before installing it on the user's device. The second solution consists of modifying the Android OS and monitoring kernel level activities to intercept malicious operations and provide more robustness to the platform. The third solution consists of using a hypervisor virtualization to implement isolated domains and run multiple instances of Android on the same device [14]–[18]. This technique in Android is known as Trustzone [19].

However, many challenges plague these solutions. For instance, the definition of malicious behavior is hard to ascertain due to the variaty of existing malware. Dynamic analysis

techniques are implemented to overcome static analysis failures such as malicious payload activities. They observe app's behaviors and analyze its features by running the application in a virtual environment such as emulators or separated sandboxes. Information collected during runtime such as system calls and APIs plays a crucial role in investigating the app's intentions. Thus, dynamic analysis is widely utilized in detecting malware samples. Yet, many samples are capable to identify the presence of the analysis environment using evasion techniques, which allow them to hide their malicious activities.

Another challenge is that, security efforts that attempt to enhance solutions by modifying the Android OS and the underlying security methodology lead to significant usability issues due to the frequently updates by Google. Moreover, the diversity of devices' hardware configurations makes those solutions tedious and costly. Thus, they hinder other efforts from worldwide adoption [20].

Therefore, in this paper, we consider overcoming these challenges by installing inspected applications on actual devices and applying some analysis techniques to collect their information during the runtime without any modifications to the device OS, as demonstrated in Section IV. Our solution extracts the inspected applications' contents such as XML layout files, resources, and classes.dex during the static analysis phase. Whereas, dynamic analysis is utilized to ensure the presence of any threatening note loaded during the runtime. Hence, this approach does not require any taint analysis or monitor the execution flow. After collecting the applications' information, it will be compared to a set of ransomware resources to measure the similarity and classify the application.

IV. Implementation

This section illustrates the unorthodox design of RanDroid framework and details its implementation. RanDroid is designed with the aim of detecting ransomware attacks in an efficient and effective manner. It focuses on detecting locking and threatening ransomware variants by investigating the appearance of locking screens and threatening notes. This framework performs its analysis task on the server side to keep the user side as efficient as possible. Users can submit applications either through RanDroid web interface, which is part of our future work, or directly from their devices through a dedicated app. This application is used as a portal to upload an APK by allowing users to browse for their desired applications and submit them to RanDroid server side for analysis before installing them on their devices. In addition, this application portal will also be used to display the analysis results. Figure 2 illustrates the overall architecture of RanDroid. The RanDroid framework consists of four main stages to be explained in the following subsections IV-A, IV-B, IV-C, and IV-D.

A. Decompilation and Extraction

After submitting an APK to the server side, RanDroid verifies the presence of the APK if it has been already analyzed by checking the back-end database where all generic results of inspected applications are stored. If exist, the result report will be retrieved and sent back to the user. Otherwise, the APK will be forwarded to the first stage of RanDroid framework, which is called Decompilation and Extraction. This stage consists of two parts as follows:

Static Analysis: in this part, RanDroid statically analyzes the APK by decompiling it using APKtool [21], which is used to disassemble and convert the APK contents to a readable format. After that, RanDroid extracts app's contributed information such as images and texts from the XML layout files, resources, and classes.dex. For this purpose, we implemented two extractor classes as follows: 1) Text Extractor, which extracts all string statements from the XML layout and classes.dex files; and 2) Image Extractor where all images in the disassembled APK are extracted and saved in one folder before sending them to the second stage.

Dynamic Analysis: while static analysis is performed, RanDroid utilizes DroidBot [22], a lightweight UI-guided test input generator to interact with an application without instrumentation. It is used to trigger app's events and captures all activities that appear while the application is running to reveal the extortion activities and investigate the existence of any threating notes or locking screens. Collected images are saved in the same folder as static analysis does.

B. Preprocessing Extracted Information

In order to detect ransomware accurately, RanDroid applies a preprocessing unit to standardize all extracted images and texts into the standard that is used in the measurement stage. To perform that, we implemented two subunits as follows. In the first subunit, RanDroid reformats images by resizing them and converting their colors into black-and-white for better identifying image contents before sending them to the next stage. In addition, ransomware images usually contain texts on them such as instructions on how to pay the ransom. Thus, RanDroid utilizes PyTesser [23], which is an optical character recognition module for Python (OCR) to extract texts applied on the images. The extracted texts will be forwarded to the second subunit to be formatted.

The second subunit is used to standardize the extracted texts from images, XML and classes.dex files into one format (i,e. lowercase with font type: Times New Roman). Furthermore, counting the frequency usage of each word in the extracted texts is performed to enhance the detection rate. Counting word frequency can help indicate the main purposes of the inspected sample.

C. Similarity Measurement

This component plays a critical role in the entire framework where the similarity measurement techniques are preformed. The inputs to this twofold stage are divided into two parts as mentioned in Section IV-B. Each part is handled by its corresponding measurement technique, which are: 1) Image Similarity Measurement (ISM); and 2) String Similarity Measurement (SSM).

In ISM, the input images are compared to 1500 extorting images. Images are collected from 850 ransomware variants

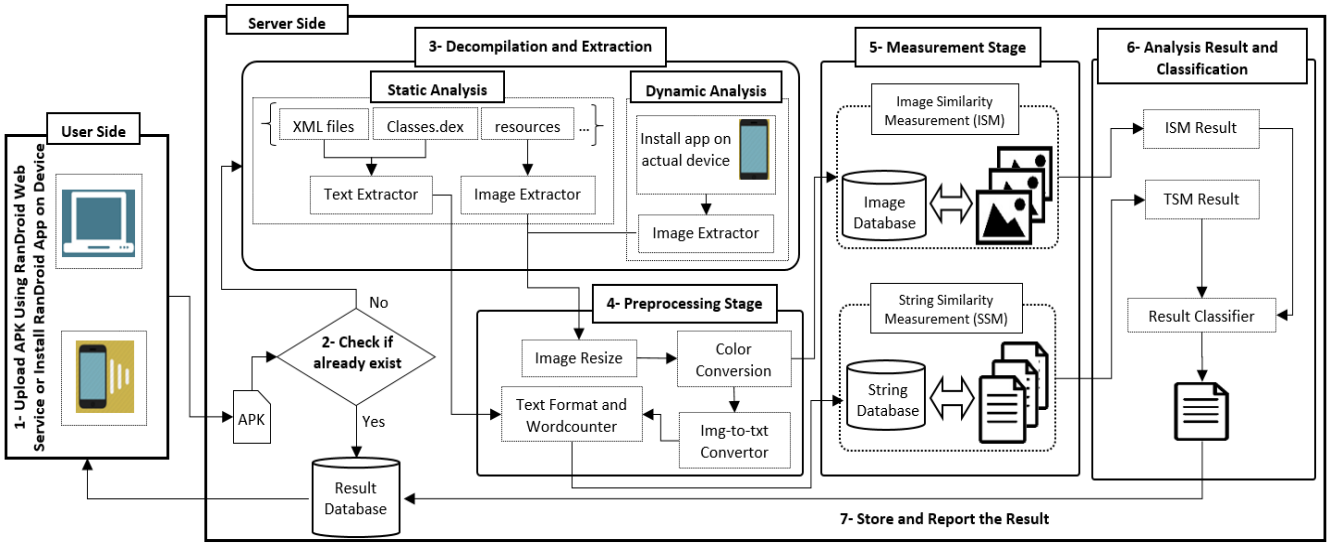

Figure 2: RanDroid architecture

and saved in a database attached to the ISM. The output of this technique is a similarity score between 0 and 1. Moreover, we use Structural Similarity Image Metric [24] by utilizing pyssim [25] with some modifications applied into it. we consider image resolution and quality, which mainly affects the result of this technique. Therefore, we set a threshold value. If the score is above the threshold value, that means the inspected image is recognized as a threat. Hence, this image is labeled as a threatening image.

On the other hand, extracted texts in the Text Extractor are forwarded to the SSM in order to compare them to a set of extorting words stored in the SSM database. This database contains more than 100 pieces of strings initialized as full sentences such as “you are hacked”, “your device is locked”, “your files are encrypted”, etc. and single words such as “encrypted”, “locked”, “ransom”, “bitcoin”, “payment”, etc. For more investigation in this part, RanDroid counts the number of matched words before forwarding the result to the next stage. If matches are found, SSM labels the string as a threatening note.

D. Analysis Results and Classification

Both ISM and SSM results are forwarded to the final stage for classifications. In this part, the results’ classifier classifies the inspected sample as a benign if and only if the results of ISM and SSM are equal to 0. If one of them has labeled its result with 1 and the other one label its result with 0, the sample is considered as a suspicious variant, which needs more investigation. Otherwise, the sample is considered as a ransomware variant.

After classifying the inspected sample, the result is stored in the back-end database. Additionally, users can obtain a copy of the result to inform themselves about the potential ransomware violations that are detected by RanDroid. In case of ransomware apps, the information which the users receive about the apps is comprised of the detected threatening notes and images as well as screenshots. All performed results can be obtained based on the sample submission preference either through the dedicated app or the web interface.

V. Discussion and Evaluation

A. Experimental Setup

Advanced malware variants are capable of identifying the presence of analysis environments using modern evasion techniques. Therefore, the inspected application is installed on real Android devices for analysis. Our framework was developed using Ubutu 16.04 LTS running on two desktop computers. The first one is used for implementing and coding RanDroid components, whereas the other computer served as a server. Furthermore, Android devices used during the dynamic analysis are attached to this server. Those devices are four Nexus 7 tablets running Android 6.0.1 Marshmallow, which gains the highest distribution among other Android OS versions [26].

The Ground Truth of our datasets are as follows:

- Ransomware: the ransomware dataset contains 950 variants collected from different resources available on the Internet including but not limited to [27]–[29]. We use 850 samples to build the ISM database where 100 samples are used to test RanDroid. However, due to the limitation on the availability of Android ransomware, we had to built few samples to test the RanDroid framework against different font sizes and styles.
- Goodware: this dataset contains about 500 applications collected from Google Play Store [30] with the assumption that all applications in this official market are benign. In building this dataset, we considered various application categories including, but not limited to, entertaining, educational, and financial applications. Moreover, we mainly focused on applications that require Internet connection

and contain more images and texts to test our dynamic analysis part against dynamic payloads. Likewise, we considered applications that behave in a way that is closer to ransomware such as screen and photo keypad locking applications.

B. Evaluation and Discussion

To calculate the accuracy rate of RanDroid, we use the following metric:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP refers to the number of ransomware samples that are correctly identified as ransomware, FN refers to the number of ransomware that are misclassified, TN refers to the number of goodware that are correctly identified as goodware, and FP refers to the number of goodware that are misclassified. Finally, the Accuracy is the ratio of total number of samples that are correctly identified as ransomware or goodware.

RanDroid was tested by running 300 applications, including 100 ransomware and 200 goodware applications. It was able to achieve 91% accuracy rate. Furthermore, misclassified ransomware (false negative (FN)) caused by samples that contain misspelled texts ot texts written in foreign languages. Whereas, misclassified goodware (false positive(FP)) caused by samples that contain large amount of texts such as books and magazines. Both FN and FP are considered as suspicious samples, which are considered as part of our future work.

The result above was obtained after running multiple tests. In the first attempt, RanDroid misclassified a few samples. After investigating this issue, we found that the misclassified results came from the ISM due to failures in finding matches to images with different resolutions. Whereas, the SSM triggered false results when the extracted texts are misspelled or displayed in different font styles. Another reason that causes FP is from samples that contain foreign language texts. Therefore, those samples are considered as suspicious and need more investigation as mentioned in our future work.

VI. Related Work

Mitigating ransomware threats attracted much attention of research efforts in the past few years. However, Android ransomware is a rather novel topic where few works have been proposed [31]–[37]. Analysts cover behaviors from various aspects such as monitoring I/O operations, recoding activities, and extracting features and API information.

Song et al. [31] introduced a technique that dynamically monitors I/O operations to the file systems. It uses statical methods based on the CPU and I/O usage to monitor abnormal processes such as file events that occurred when an operation attempts to access and copy files. Yet, this technique fails against variants that perform locking and threatening texts. Another proposed solution for detecting crypto-ransomware attempts is Sdguard [35], which implements fine-grain permission control based on Linux Discretionary Access Control (DAC) mechanism. Sdguard monitors activities in the Android activity stack and the access control list using FUSE file system (Filesystem in Userspace), which contains a permission checker module to verify permissions prior determining an access. Moreover, Sdguard uses the FUSE to record all I/O operations to investigate malicious behavior existence using a log analyzer.

Furthermore, DNA-Droid framework [32] utilizes features and deep neural network to discriminate ransomware actions in an early stage by extracting app's static features and label it. DNA-Droid applies dynamic analysis to monitor run-time behaviors by matching them with a set of collected malicious behaviors. Another prominent approach is called HelDroid [33], which combines taint analysis with a symbolic execution to find paths that indicate ransomware-like behaviors. It detects threatening texts by utilizing a text classifier that applies linguistic features. However, HelDroid is vulnerable to samples that use evasion mechanisms or embedding cryptographic primitives. In addition, a notable work in Android ransomware detection based API packages is R-PackDroid [34]. This approach demonstrates the possibility of detecting ransomware based on an extracted API package information.

However, in RanDroid, we did not consider monitoring applications' processes such as I/O operations or applying taint analysis. Such techniques can possibly produce a high overhead, which we aim to avoid in our approach. Furthermore, we target locking and threating variants by monitoring information that indicate their appearance such as images and extorting texts. This novel technique identifies ransomware applications with low overhead. In RanDroid approach, it is alarming to see the potential that locking and threatening ransomware variants can adopt evasion techniques to bypass detection systems to achieve their goals.

VII. Conclusion and Future Work

This paper introduced RanDroid, an automated lightweight solution for mitigating the worldwide epidemic of locking and threatening ransomware on Android platform. This proposed framework utilizes static and dynamic analyses combines to extract information from inspected applications. Combining static and dynamic analyses with comprehensive functionalities in RanDroid has fostered ransomware threat mitigation without modifying the Android OS and the underlying security methodology. Furthermore, RanDroid measures the structural similarity between a set of collected images and texts and a set of predefined threatening ones collected from known ransomware variants. In addition, this approach is capable of extracting threatening messages from sample that use evasion techniques such as code's sophistication or dynamically payloads.

To increase RanDroid accuracy, more samples will be added into our dataset to increase the number of images and extorting strings in the ISM database and the SSM database respectively. The new images will contain governments' logos and other low reinforcement agencies' icons in order to detect variants that use false alerts such as fake FBI notes. Additionally, text recognition techniques such as Natural-language processing

(NLP) will be applied to enhance the SSM to accurately recognize extorting texts. In particular, we will apply spelling auto-correction techniques to overcome miss-spelled texts and decrease the error rate caused by PyTesser when extracting texts from the images.

ACKNOWLEDGMENT

This research work is partially supported by the National Science Foundation under Grants CNS-1460897, DGE-1623713, and DGE-1723707. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

REFERENCES

[1] I. D. Corporation. (2017) Smartphone os market share, 2017 q1. [Online]. Available: http://www.idc.com/promo/smartphone-market-share/os

[2] Bromium. (2017) Understanding crypto-ransomware. [Online]. Available: https://www.bromium.com/sites/default/files/rpt-bromium-crypto-ransomware-us-en.pdf

[3] U. Bayer, I. Habibi, D. Balzarotti, E. Kirda, and C. Kruegel, "A view on current malware behaviors," in *LEET*, 2009.

[4] Q. Yan, F. R. Yu, Q. Gong, and J. Li, "Software-defined networking (sdn) and distributed denial of service (ddos) attacks in cloud computing environments: A survey, some research issues, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 602–622, 2016.

[5] G. Suarez-Tangil, J. E. Tapiador, P. Peris-Lopez, and A. Ribagorda, "Evolution, detection and analysis of malware for smart devices," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 2, pp. 961–987, 2014.

[6] A. Kharraz, S. Arshad, C. Mulliner, W. Robertson, and E. Kirda, "Unveil: A large-scale, automated approach to detecting ransomware," in *25th USENIX Security Symposium (USENIX Security 16)*. USENIX Association, 2016, pp. 757–772.

[7] Advance2000. (2015) Security alert–ransomware. [Online]. Available: http://www.advance2000.com/wp-content/uploads/2017/02/Ransomware-info.pdf

[8] J. Wyke and A. Ajjan. (2015) The current state of ransomware. [Online]. Available: https://www.sophos.com/en-us/medialibrary/PDFs/technical%20papers/sophos-current-state-of-ransomware.pdf

[9] N. Scaife, H. Carter, P. Traynor, and K. R. Butler, "Cryptolock (and drop it): stopping ransomware attacks on user data," in *Distributed Computing Systems (ICDCS), 2016 IEEE 36th International Conference on*. IEEE, 2016, pp. 303–312.

[10] A. Kharraz, W. Robertson, D. Balzarotti, L. Bilge, and E. Kirda, "Cutting the gordian knot: A look under the hood of ransomware attacks," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2015, pp. 3–24.

[11] R. Vogt, J. Aycock, and M. J. Jacobson Jr, "Army of botnets," in *NDSS*, 2007.

[12] H. Orman, "Evil offspring-ransomware and crypto technology," *IEEE Internet Computing*, vol. 20, no. 5, pp. 89–94, 2016.

[13] E. Kolodenker, W. Koch, G. Stringhini, and M. Egele, "Paybreak: Defense against cryptographic ransomware," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*. ACM, 2017, pp. 599–611.

[14] D. J. Tan, T.-W. Chua, V. L. Thing *et al.*, "Securing android: a survey, taxonomy, and challenges," *ACM Computing Surveys (CSUR)*, vol. 47, no. 4, p. 58, 2015.

[15] B. Reaves, J. Bowers, S. A. Gorski III, O. Anise, R. Bobhate, R. Cho, H. Das, S. Hussain, H. Karachiwala, N. Scaife *et al.*, "* droid: Assessment and evaluation of android application analysis tools," *ACM Computing Surveys (CSUR)*, vol. 49, no. 3, p. 55, 2016.

[16] J. Andrus, C. Dall, A. V. Hof, O. Laadan, and J. Nieh, "Cells: a virtual mobile smartphone architecture," in *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*. ACM, 2011, pp. 173–187.

[17] M. Lange, S. Liebergeld, A. Lackorzynski, A. Warg, and M. Peter, "L4android: a generic operating system framework for secure smartphones," in *Proceedings of the 1st ACM workshop on Security and privacy in smartphones and mobile devices*. ACM, 2011, pp. 39–50.

[18] K. Gudeth, M. Pirretti, K. Hoeper, and R. Buskey, "Delivering secure applications on commercial mobile devices: the case for bare metal hypervisors," in *Proceedings of the 1st ACM workshop on Security and privacy in smartphones and mobile devices*. ACM, 2011, pp. 33–38.

[19] A. Developer. (2018) Trusty tee. [Online]. Available: https://source.android.com/security/trusty/

[20] L. Wei, Y. Liu, and S.-C. Cheung, "Taming android fragmentation: Characterizing and detecting compatibility issues for android apps," in *Automated Software Engineering (ASE), 2016 31st IEEE/ACM International Conference on*. IEEE, 2016, pp. 226–237.

[21] APKTool. (2012) Android–apktool: A tool for reverse engineering android apk files. [Online]. Available: https://ibotpeaches.github.io/Apktool/

[22] Y. Li, Z. Yang, Y. Guo, and X. Chen, "Droidbot: a lightweight ui-guided test input generator for android," in *Software Engineering Companion (ICSE-C), 2017 IEEE/ACM 39th International Conference on*. IEEE, 2017, pp. 23–26.

[23] P. S. Foundation. (2018) Pytesser 0.0.1. [Online]. Available: https://pypi.python.org/pypi/PyTesser/

[24] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.

[25] J. Terrace. (2018) pyssim. [Online]. Available: https://github.com/jterrace/pyssim/blob/master/README.md

[26] A. Developer. (2018) Dashboards. [Online]. Available: https://developer.android.com/about/dashboards/index.html

[27] Encrypting-leeches and ID-Ransomware.RU. (2016) Encrypting programs-extortionists. [Online]. Available: https://id-ransomware.blogspot.ru/2016/07/ransomware-list.html

[28] Contagio. (2017) Malware dump. [Online]. Available: http://contagiodump.blogspot.com/

[29] Y. Nativ. (2017) thezoo aka malware db. [Online]. Available: http://thezoo.morirt.com/

[30] G. Play. (2018) Google play store. [Online]. Available: https://play.google.com/store

[31] S. Song, B. Kim, and S. Lee, "The effective ransomware prevention technique using process monitoring on android platform," *Mobile Information Systems*, vol. 2016, 2016.

[32] A. Gharib and A. Ghorbani, "Dna-droid: A real-time android ransomware detection framework," in *International Conference on Network and System Security*. Springer, 2017, pp. 184–198.

[33] N. Andronio, S. Zanero, and F. Maggi, "Heldroid: Dissecting and detecting mobile ransomware," in *International Workshop on Recent Advances in Intrusion Detection*. Springer, 2015, pp. 382–404.

[34] D. Maiorca, F. Mercaldo, G. Giacinto, C. A. Visaggio, and F. Martinelli, "R-packdroid: Api package-based characterization and detection of mobile ransomware," in *Proceedings of the Symposium on Applied Computing*. ACM, 2017, pp. 1718–1723.

[35] S. Hong, C. Liu, B. Ren, and J. Chen, "Poster: Sdguard: An android application implementing privacy protection and ransomware detection," in *Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services*. ACM, 2017, pp. 149–149.

[36] J. K. Lee, S. Y. Moon, and J. H. Park, "Cloudrps: a cloud analysis based enhanced ransomware prevention system," *The Journal of Supercomputing*, vol. 73, no. 7, pp. 3065–3084, 2017.

[37] F. Mbol, J.-M. Robert, and A. Sadighian, "An efficient approach to detect torrentlocker ransomware in computer systems," in *International Conference on Cryptology and Network Security*. Springer, 2016, pp. 532–541.